Sustainable Resource Allocation in Urban School Settings (STRAUSS): RQ1 Formalism

January 21, 2026

1 Objects

- 3 disjoint sets; students, colleges, routes:

  $S = \{s_1, ..., s_{n_S}\}, C = \{c_1, ..., c_{n_C}\}, R = \{r_1, ..., r_{n_R}\}$.

- 2 sets constructed from elements of the 3 sets above; route-college tuples and route-student tuples:

  $A = \{a_{11}, ..., a_{n_R n_C}\}$ where eg. for student $s_1 : a_{11} = (r_1, c_1) \succ ... \succ a_{23} = (r_2, c_3) \succ ...$ etc,

  $B = \{b_{11}, ..., b_{n_R n_S}\}$ where eg. for college $c_1 : b_{11} = (r_1, s_1) \succ ...$, for $c_3 : b_{21} = (r_2, s_1) \succ ...$, etc.

- College over-subscription priorities divided into 2 priority brackets given by:

  Students *within* a certain distance of a school selecting that school **without** transport AND students *within* a selected area served by a route, selecting a school **with** transport are placed in the same priority bracket; all other students placed in the bracket below this.

- Routes can have a capacity of more than 1: $q_r \geq 1$.

- Each route serves one college. Routes are effectively taxi services.

- Routes are only available in selected areas (only feasible for a subset of students $S_{r_i} \in S$ for a route i). The area would be selected based on SES indicators, such as eligibility for free school meals, to maximise the impact of improving priority for students (current model selects a random subset of students to be served by routes, and each student is served by one route).

- Eligible students (those served by routes) can select either a college, or route-college tuple, eg. in student s_1's preference list: $a_{11} = (r_1, c_1) \succ a_{23} = (r_2, c_3) \succ c_1 \succ c_3 \succ ...$ etc. Each object a contains exactly one college. Ineligible students may only select colleges and are unable to select route-college tuples.

- Each route-college tuple has a **corresponding route-student tuple**, where the route is shared between the two tuples, eg. for $c_1 : b_{11} = (r_1, s_1), s_1, ...$, and for $c_3 : b_{21} = (r_2, s_1), s_1, ...$, etc from the route-college tuples above. All route-college tuples have their corresponding route-student tuples added to the top priority bracket in the priority list of the college in the route-college tuple.

- Student preferences over colleges and route-college tuples (that are available to them), and college priorities over students and route-student tuples are strict and complete.

2 Algorithm (modified deferred acceptance)

1. Start with no student or college assigned to a matching: $\mu = \emptyset$.
2. While some student s_n is free:
   (a) s_n proposes to the first preference on their preference list (either c_j or $a_{ij} = (r_i, c_j)$):
   (b) If applying to a college with a route, a_{ij}, then $\mu = \mu \cup \{s_n, a_{ij}\}$. If route r_i has reached capacity:
      i. The student s_k from the lowest priority, complementary route-student tuple $b_{ik} = (r_i, s_k)$ competing for the route r_i in the priority list for c_j is released from the matching: $\mu = \mu \backslash \{s_k, a_{ij}\}$. Since each route only serves one college, the student s_k is also rejected from the associated college c_j. Student s_k removes a_{ij} from their preference list and will not propose to them again.

(c) If applying to a college c_j with no route then $\mu = \mu \cup \{s_n, c_j\}$. If applying to a college with a route a_{ij} then $\mu = \mu \cup \{s_n, a_{ij}\}$. If the college is oversubscribed in either case:

i. The lowest priority object (either student s_k or student-route tuple $b_{il} = (r_i, s_l)$) in the priority list for c_j is identified:

- in the case of s_k, it is directly released from the matching: $\mu = \mu \backslash \{s_k, c_j\}$, and s_k removes c_j from their preference list and will not propose to them again,
- in the case of b_{il} the complementary object $a_{ij} = (r_i, c_j)$ and student s_l are released from the matching: $\mu = \mu \backslash \{s_l, a_{ij}\}$. If b_{il} is released from c_j then s_l is also released from the route r_i. Student s_l will remove a_{ij} from their preference list and will not propose to them again.

(d) If any student s_n has applied to all objects in their preference list, and has been rejected by all of them, thus exhausting their preference list, they issue no further applications and are no longer considered "free," but "unmatched."

3 Checking stability

The algorithm outputs two objects: the (stable) matching and the list of unassigned students; both must be checked for blocking pairs.

1. For the matching:
   (a) For each student, s, check their top preference:
   - IF they are assigned their top preference then they cannot form a blocking pair.
   - ELSE IF they are not assigned their top preference, begin checking the preferences starting with the most preferred:
     - IF the preference has no route associated with it AND the associated college has spare capacity: forms a blocking pair.
     - ELSE IF the preference has no route associated with it AND the associated college IS at capacity BUT the college prefers s to any of its assigned students: form a blocking pair.
     - ELSE IF the preference does have a route associated with it, which is NOT at capacity AND the college has spare capacity: form a blocking pair.
     - ELSE IF the preference does have a route associated with it, which is NOT at capacity AND the college IS at capacity BUT the college prefers s over any of its assigned students: form a blocking pair.
     - ELSE IF the preference does have a route associated with it which IS at capacity AND the college is at capacity OR has spare capacity AND s is preferred to any routed student: form a blocking pair.
2. For the list of unassigned students (if it exists):
   - IF the preference has no route associated with it AND the associated college has spare capacity: forms a blocking pair.
   - ELSE IF the preference has no route associated with it AND the associated college IS at capacity BUT the college prefers s to any of its assigned students: form a blocking pair.
   - ELSE IF the preference does have a route associated with it, which is NOT at capacity AND the college has spare capacity: form a blocking pair.
   - ELSE IF the preference does have a route associated with it, which is NOT at capacity AND the college IS at capacity BUT the college prefers s over any of its assigned students: form a blocking pair.
   - ELSE IF the preference does have a route associated with it which IS at capacity AND the college is at capacity OR has spare capacity AND s is preferred to any routed student: form a blocking pair.

4 Proof of termination

1. The number of agents, the length of the preference and priority lists, and the college and route capacities are all finite.

2. Students propose to objects in order of preference, and if they are accepted by an object they will not propose again unless they are unmatched with that object at a later stage. If they are unmatched later, they will not propose to objects that they have been unmatched with, so their preference lists will be weakly decreasing with every iteration of the algorithm.
3. Even if all students were unacceptable to all colleges, and every student was rejected at every stage, there would be a maximum of $n \times m$ proposals, where n is the number of students and m is the number of colleges.

5 Proof of stability

Suppose there exists some student s and [college c OR tuple (r, c)] which are not matched together in μ after termination of the algorithm such that neither (s, c) nor $(s, (r, c))$ are in the matching μ.

Suppose also that s prefers [c OR (r, c)] to $\mu(s)$ and hence [c OR (r, c)] must be acceptable to s, and s must have applied to [c OR (r, c)] before applying to $\mu(s)$. In the case where $\mu(s)$ is empty, the student s must be unmatched, and hence must have exhausted their entire preference list before ending up unmatched.

Since s is not matched to [c OR (r, c)] when the algorithm terminates:

- IF s is NOT eligible for r: either s applied to c, was accepted and then later rejected by c in favour of a higher priority student/routed student, OR c was already at capacity at the point of application and was rejected as the lowest priority student/routed student.
- ELSE IF s IS eligible for r and ONLY applied to (r, c); either:
  - s applied to (r, c), was accepted and was later rejected from r (and since r only serves c is therefore also rejected from c) in favour of a higher priority student routed to c, OR r was already at capacity at the point of application and s was rejected as the lowest priority student routed to r,
  - OR
  - s applied to (r, c), was accepted and later rejected from c in favour of a higher priority student/routed student, OR c was already at capacity at the point of application and was rejected as the lowest priority student/routed student.
- ELSE IF s IS eligible for r and applied to BOTH (r, c) AND c; BOTH:
  - s applied to (r, c), was accepted and was later rejected from r (and therefore also c) in favour of a higher priority routed student, OR r was already at capacity at the point of application and s was rejected as the lowest priority routed student
  - AND
  - s applied to c, was accepted and was later rejected by c in favour of a higher priority student/routed student, OR c was already at capacity at the point of application and was rejected as the lowest priority student/routed student.

A key aspect of the stability of our algorithm is the order in which the oversubscription is resolved in the case where both college and route are oversubscribed: as long as route oversubscription is resolved first, college oversubscription is resolved simultaneously and the number of remaining seats of a college *can only weakly decrease with each step of the algorithm.*

Because of this, *the lowest priority student in a college's preference list can* ***only*** *weakly improve.* Hence, the cases above hold not only at the point of rejection, but at the termination of the algorithm.

Additionally, matchings may be stable, even if routes are undersubscribed.

Below are examples that showcase the above statements:

5.1 Examples

5.1.1 Case 1

Consider the case:

College	Priorities
c_1	$(r_1, s_4), (r_1, s_5), (r_1, s_6), s_3, (r_1, s_7)$

where c_1 has a capacity of 3 and r_1 has a capacity of 2.

If the students propose in the reverse order of college c_1's priority list, then we see that:

1. $(r_1, s_6), s_3, (r_1, s_7)$ are initially assigned to c_1,
2. (r_1, s_5) applies and displaces (r_1, s_7) since r_1 has reached capacity (as well as c_1) and (r_1, s_7) is the lowest priority:
   - r_1 and c_1 reach capacity simultaneously, but since the lowest priority item, (r_1, s_7), is simultaneously the lowest priority item in c_1's priority list *and* the lowest priority routed student, the order in which the overcapacity problem is solved does not matter.
3. (r_1, s_4) applies and now there is a problem: both r_1 and c_1 are already at capacity, and the order in which the oversubscription is resolved is important:
   - If c_1's oversubscription is resolved first, then s_3 will be rejected from the college, after which (r_1, s_6) will be rejected from both route and college, resulting in the final allocation $c_1 : (r_1, s_4), (r_1, s_5)$, leaving the college *undersubscribed*, but importantly violating one of the principles that guarantees termination and stability: that colleges only "trade up."
   - On the contrary, if the route oversubscription is resolved first, then only (r_1, s_6) will be rejected from both route and college, which satisfies both capacity conditions simultaneously, and the final allocation will be: $c_1 : (r_1, s_4), (r_1, s_5), s_3$.

5.1.2 Case 2

Consider the next case where the priority list is prepended with s_1, s_2, who are local students in the upper priority bracket.

College	Priorities
c_1	$s_1, s_2, (r_1, s_4), (r_1, s_5), (r_1, s_6), s_3, (r_1, s_7)$

After following the same procedure as above, s_1, s_2 now apply to c_1 in reverse order as before, displacing first s_3, then (r_1, s_5). This will leave r_1 *undersubscribed*. This is not an issue here however because there are no students, routed or otherwise who have a higher priority than the assigned students, and that would be able to change the assignment, as all other acceptable students have already been rejected, and they cannot re-apply later.

6 Proof of Student Optimality

Theorem: No student is rejected by an achievable object and so the algorithm returns the student optimal stable matching.

Proof: Suppose the algorithm is at iteration i and no student has been unmatched with an object that is achievable to them:

1. At this iteration i suppose route r, which serves only college c and inherits its priority list, is at capacity and student s who applied to c with route r (henceforth called (r, s)), is rejected from both r and c:
   - If (r, s) is rejected in favour of (r, s'), then (r, s) is lower in c's priority list than (r, s').
   - Since there have been no rejections up until this iteration, and students apply to their top preferences first in the algorithm, we know that (r, s') prefers c over all other colleges.
   - If we consider a matching μ where (r, s) is matched to c and all other students are matched to achievable objects, we know it will be unstable since (r, s') prefers c to its current partner, and c prefers (r, s') to (r, s).
   - Hence, there is no stable matching where (r, s) is matched to c, and so they are unachievable for each other.
2. Else at this iteration suppose college c is at capacity and rejects student s or (r, s).
   - If s or (r, s) is rejected in favour of s' or (r, s'), then s or (r, s) is lower in c's priority list than either of s' or (r, s').
   - Clearly s' or (r, s') prefer c over all other colleges since students apply to their top preferences first in the algorithm, and no student has been rejected as of iteration i.
   - If we consider a matching μ' where s or (r, s) is matched to c and all other students are matched to achievable objects, we know it will be unstable since s' or (r, s') prefers c to its current partner, and c prefers s' or (r, s') to s or (r, s).
   - Hence, there is no stable matching where s or (r, s) is matched to c, and so they are unachievable for each other.